Migration to a Service-Based Architecture

Status
Proposed

Context
The Spring Petclinic application is a well-structured monolith. Analysis of the source code reveals distinct, domain-oriented packages: owner, vet, and model (which contains the core entities for visits). While this modular monolith has been effective, it presents future challenges:

Scalability: The entire application must be scaled together. If, for example, the appointment booking (visits) functionality experiences high load, we must deploy more instances of the entire application, which is resource-intensive.
Maintainability: Any change, even within a single domain like vet, requires a full re-deployment of the entire application, increasing risk and slowing down the development cycle.
Technology Stack: The entire application is tied to a single technology stack (Java/Spring Boot). We cannot introduce new technologies for specific domains (e.g., using a different database for a new feature) without significant refactoring.

Decision
We will refactor the PetClinic monolith into a service-based architecture, starting with the most distinct and high-traffic domains. This is a strategic step towards a full microservices architecture.

Based on the codebase's domain-driven structure, we will extract the following as initial, independent services:

1. **Vets Service**: Manages veterinarians and their specialties (derived from the vet package). This service is largely read-only and has minimal dependencies on other domains, making it an ideal first candidate.
2. **Visits Service**: Manages pet visit information (derived from the Visit entity in the model package). This is a high-transaction area of the application and would benefit most from independent scaling.
3. **Customer Service**: Manages pet owners and their pets (derived from the owner package). This is the core service and will initially retain the remaining functionality.

These services will communicate via a well-defined REST API, and an API Gateway will be introduced to route incoming requests.

Consequences

Positive

- **Improved Scalability**: We can independently scale the Visits Service during peak appointment times without scaling the entire application.
- **Enhanced Maintainability**: A change to the vet directory (Vets Service) can be deployed independently without affecting the core customer and visit functionality. This reduces deployment risk and time.
- **Clear Ownership**: Teams can take ownership of specific services, leading to faster development and expertise.

Negative

- **Increased Complexity**: We will need to manage inter-service communication, service discovery, and data consistency between services.
- **Operational Overhead**: Requires a more robust CI/CD pipeline, centralized logging, and monitoring to manage the distributed nature of the application.

Risks

- **Data Consistency**: A pet owner might be deleted from the Customer Service while a visit is being scheduled in the Visits Service. We will need to implement strategies like eventual consistency or two-phase commits to mitigate this.
- **Network Latency**: Direct method calls within the monolith will be replaced by network calls, which can introduce latency. The API design must be efficient.

Costs

- **Infrastructure Costs**: Additional costs for hosting the new services, running an API Gateway, and a service registry.
- **Development Costs**: The initial refactoring effort is significant and will require dedicated developer time.
- **Training Costs**: The team will need to be trained on distributed systems principles, API design, and new operational tools.